AGENTS

INHERITED FROM Helix Constitution

> Base agent rules live in the Helix Constitution submodule at the parent project's `constitution/AGENTS.md` and the universal `constitution/Constitution.md` it references. **READ THOSE FIRST.** The base file is authoritative for any topic not covered here. Module-specific rules below extend them; they never weaken them.

Critical universal rules every CLI agent (Claude Code, Cursor, Aider, Codex, Gemini CLI) MUST honour while working in this module:

- **No bluffing.** Every PASS carries positive evidence. Constitution §11.4.
- **Mutation-paired gates.** Every new gate has a paired mutation proving it catches regressions. Constitution §1.1.
- **No guessing language** (`likely`, `probably`, `maybe`, `seems`). Constitution §11.4.6.
- **Credentials never tracked.** `.env` patterns git-ignored; runtime-load only. Constitution §11.4.10.
- **Never force-push.** Force-push requires explicit per-session authorization AND a green §9.1.5 post-op gate. Constitution §9.
- **CONTINUATION.md kept in sync** in every non-trivial commit. Constitution §12.10.
- **60% RAM cap.** Heavy work wrapped in bounded execution scope. Constitution §12.6.

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

---

AGENTS.md — HelixCode Authoritative Agent Guide

> **Base agent rules:** `HelixConstitution/AGENTS.md` — READ IT FIRST. All rules in `HelixConstitution/AGENTS.md` apply unconditionally. Rules below extend them and MUST NOT weaken any inherited clause.

HelixCode Agent Guidelines

Version: 3.0.0 (Updated with full architecture audit) **Date**: 2026-04-30 **Scope**: All AI agents, human contributors, and automated processes working on HelixCode **Authority**: Derived from HelixAgent AGENTS.md with HelixCode-specific enhancements

---

Project Overview

HelixCode is an enterprise-grade distributed AI development platform built in Go. It enables intelligent task division, work preservation, cross-platform development workflows, and multi-provider LLM integration through a unified REST API, CLI, Terminal UI, Desktop, and Mobile client architecture.

Current Status: The `internal/` foundation is largely solid (auth, database, server, worker, task, workflow, tools, editor, notification, MCP, **verifier** are real implementations). Critical bluff and stub areas remain in select entry points and peripheral packages. All agents MUST prioritize zero-bluff implementation.

LLMsVerifier Integration Status: `internal/verifier/` package is now implemented with REST API client, two-tier cache, circuit breaker health monitor, background poller, score adapter, and event publisher. BLUFF-002 (hardcoded CLI models) and BLUFF-004 (hardcoded external models) are FIXED. BLUFF-005 (scoring ignores verifier data) is FIXED in `ModelManager.SelectOptimalModel()`.

Key Features:

- **Distributed Computing**: SSH-based worker pools with health monitoring, auto-installation, and consensus
- **Multi-Provider LLM Integration**: 15+ providers (OpenAI, Anthropic, Gemini, Ollama, Azure, Bedrock, Groq, Mistral, Cohere, xAI, DeepSeek, Qwen, OpenRouter, HuggingFace, Llama.cpp)

- **Development Workflows**: Automated planning, building, testing, refactoring with real shell execution
- **Task Management**: Intelligent task division with priorities, dependencies, checkpointing, and Redis caching
- **MCP Protocol**: Full Model Context Protocol server over WebSocket with tool dispatch
- **Multi-Client Architecture**: REST API (Gin), Cobra CLI, Terminal UI (tview), Desktop (Fyne), Mobile (gomobile), WebSocket
- **Memory Systems**: In-memory, filesystem, Redis, Memcached, Cognee, ChromaDB, Qdrant, Weaviate integrations
- **Advanced Editor**: Multi-format code editing (diff, whole-file, search/replace, line-based) with backups
- **Tools Ecosystem**: 40+ tools across filesystem, shell, web, browser, mapping, multiedit, confirmation, notebook, git
- **Notifications**: Multi-channel support (Slack, Email, Telegram, Discord, Yandex Messenger, Max)

Technology Stack

Core Technologies:

- **Language**: Go 1.24.0 with toolchain go1.24.9
- **Module**: `dev.helix.code`
- **HTTP Framework**: Gin v1.11.0
- **Authentication**: JWT v4.5.2, bcrypt + argon2
- **Database**: PostgreSQL 15+ via pgx/v5 (optional)
- **Cache**: Redis 7+ via go-redis/v9 (optional)
- **Configuration**: Viper v1.21.0
- **CLI Framework**: Cobra v1.8.0
- **Testing**: Testify v1.11.1

UI Technologies:

- **Desktop**: Fyne v2.7.0
- **Terminal UI**: tview v0.42.0
- **Mobile**: gomobile bindings

External Integrations:

- **Browser Automation**: chromedp v0.14.2
- **Web Scraping**: goquery v1.10.3
- **Tree-sitter**: go-tree-sitter
- **Identity**: Azure SDK, AWS SDK v2
- **Vector/Memory**: Cognee, ChromaDB, Qdrant, Weaviate clients
- **Container Orchestration**: digital.vasic.containers (vasic-digital/Containers submodule)

Working Directory & Build System

CRITICAL: All build and test commands must be run from the `helix_code/` subdirectory, not the repository root.

```
cd HelixCode
```

Build Commands

Command	Purpose
`make build`	Build server binary to `bin/helixcode`
`make test`	Run `go test -v ./...`
`make test-all`	Run tests + coverage + benchmarks + docs
`make test-coverage`	Generate coverage report
`make test-benchmark`	Run Go benchmarks
`make logo-assets`	Generate logo assets (required before first build)
`make setup-deps`	Run `go mod tidy`
`make fmt`	Run `go fmt ./...`
`make lint`	Run `golangci-lint run ./...`
`make clean`	Clean build artifacts
`make dev`	Start development server
`make prod`	Cross-platform production build
`make mobile`	Build iOS + Android targets
`make aurora-os`	Build Aurora OS target
`make harmony-os`	Build Harmony OS target

Full Infrastructure Test Commands

Command	Purpose
`make test-infra-up`	Start full Docker test infrastructure
`make test-infra-down`	Stop full Docker test infrastructure
`make test-full`	ALL tests with real infrastructure (zero skips)
`make test-unit-full`	Unit tests with real services
`make test-integration-full`	Integration tests with `-tags=integration`
`make test-e2e-full`	E2E challenge tests via runner
`make test-security-full`	Security test suite
`make test-load-full`	Load tests

Command	Purpose
`make test-complete`	Sequential run of all full test types
`make coverage-full`	Coverage with full infrastructure

Containerized Builds (NO Host Dependencies)

Command	Purpose
`make container-builder-image`	Build the builder container image
`make container-build`	Build application inside container
`make container-test`	Run tests inside container
`make container-lint`	Run linter inside container
`make container-shell`	Interactive shell in builder container
`make container-dev-up`	Start containerized dev environment
`make container-dev-down`	Stop containerized dev environment
`make container-release`	Full release build in container
`./scripts/containers/build-in-container.sh`	Convenience wrapper script

The builder container includes: Go 1.24, gcc, postgresql-client, redis, docker-cli, golangci-lint, and all build tools. The only host requirement is Docker/Podman.

Standalone Test Scripts

Script	Purpose
`./run_tests.sh --unit`	Unit tests
`./run_tests.sh --integration`	Integration tests
`./run_tests.sh --e2e`	E2E tests
`./run_tests.sh --coverage`	Coverage analysis
`./run_tests.sh --security`	Security tests
`./run_all_tests.sh`	Orchestrates ALL suites sequentially
`./run_integration_tests.sh`	DB integration tests with Docker

Single Test Execution

```
go test -v -run TestName ./path/to/package
go test -v -tags=integration ./internal/database
cd tests/e2e/challenges && go run cmd/runner/main.go -challenge
        ascii-art-generator-001 -providers ollama
```

Architecture & Code Organization

```
helix_code/
├── cmd/                              # Application entry points
│   ├── server/main.go                # HTTP server entry point
│   ├── cli/main.go                   # Legacy flag-based CLI client
│   ├── root.go                       # Cobra root command (`helix`)
│   ├── main_commands.go              # `helix start`, `helix auto`
│   ├── other_commands.go             # `helix server`, `helix
version`, etc.
│   ├── local-llm.go                  # `helix local-llm` command tree
│   ├── local-llm-advanced.go         # Advanced local-llm commands
│   ├── helix-config/main.go          # Dedicated config management
CLI
│   ├── security-test/main.go         # Simulated security test runner
│   ├── security-fix/main.go          # Security fix wrapper
│   ├── security-fix-standalone/main.go  # Standalone security
scanner
│   ├── performance-optimization/main.go # Performance optimizer
│   ├── performance-optimization-standalone/main.go # Standalone
perf simulator
│   └── config-test/main.go           # Config hot-reload test utility
│
├── internal/                         # Internal packages (~40
packages)
│   ├── auth/                         # JWT authentication, bcrypt/
argon2, sessions
│   ├── llm/                          # LLM provider implementations
(15+ providers)
│   │   ├── providers/                # Per-provider HTTP clients
│   │   ├── compression/              # Context compression
│   │   └── vision/                   # Vision/multimodal support
│   ├── provider/                     # Provider abstractions
│   ├── providers/                    # Provider management
│   ├── worker/                       # SSH-based worker pool, health
checks
│   ├── task/                         # Task queues, dependencies,
checkpoints
│   ├── server/                       # Gin HTTP server, routes,
middleware
│   ├── database/                     # PostgreSQL pgx pool, schema
initialization
```

```
│   ├── redis/                    # go-redis wrapper with graceful
degradation
│   ├── tools/                    # 40+ tool ecosystem registry
│   │   ├── filesystem/           # fs_read, fs_write, fs_edit,
glob, grep
│   │   ├── shell/                # shell, shell_background with
sandbox
│   │   ├── web/                  # web_fetch, web_search
│   │   ├── browser/              # browser_launch,
browser_navigate, browser_screenshot
│   │   ├── multiedit/            # Transactional multi-file
editing
│   │   └── git/                  # Git automation
│   ├── editor/                   # Multi-format code editing with
backups
│   ├── memory/                   # Memory providers (in-mem,
filesystem, Redis, etc.)
│   ├── cognee/                   # Cognee.ai memory integration
│   ├── context/                  # Hierarchical context
management with TTL
│   ├── notification/             # Multi-channel notification
engine
│   ├── mcp/                      # Model Context Protocol
WebSocket server
│   ├── workflow/                 # Development workflow execution
│   ├── config/                   # Viper-based configuration
management
│   ├── event/                    # Pub/sub event bus
│   ├── logging/                  # Structured logging wrapper
│   ├── monitoring/               # Metric collection framework
│   ├── security/                 # Security scanning (stubbed)
│   ├── session/                  # Development session management
│   ├── agent/                    # Agent orchestration
│   ├── project/                  # Project management
│   ├── rules/                    # Rules engine
│   ├── hooks/                    # Hook system
│   ├── focus/                    # Focus chain management
│   ├── template/                 # Template system
│   ├── persistence/              # State persistence
│   ├── deployment/               # Deployment management
│   ├── discovery/                # Service/model discovery
│   ├── hardware/                 # Hardware abstraction
│   ├── repomap/                  # Repository mapping
│   ├── version/                  # Version management
│   ├── fix/                      # Security fix engine
│   ├── performance/              # Performance optimization
│   ├── testutil/                 # Test utilities
│   └── mocks/                    # Shared mocks
│
├── applications/                 # Platform-specific applications
│   ├── desktop/                  # Fyne desktop app
│   ├── terminal-ui/              # tview terminal UI
```

```
│   ├── android/                  # Android app
│   ├── ios/                      # iOS app
│   ├── aurora-os/                # Aurora OS client
│   └── harmony-os/               # Harmony OS client
│
├── api/                          # OpenAPI specification
│   └── openapi.yaml              # Full REST API spec (OpenAPI
3.0.3)
│
├── config/                       # Configuration files
│   ├── config.yaml               # Primary application config
│   ├── production-config.yaml    # Enterprise production config
│   ├── minimal-config.yaml       # Minimal test config (DB/Redis
disabled)
│   ├── test-config.yaml          # Test-specific config
│   ├── working-config.yaml       # Working variant
│   ├── azure_example.yaml        # Azure-specific example
│   └── model-aliases.example.yaml# Model alias examples
│
├── tests/                        # New test framework
│   ├── e2e/challenges/           # Challenge-based E2E tests
│   └── automation/               # Hardware automation tests
│
├── test/                         # Legacy/parallel test suites
│   ├── integration/              # Integration tests
│   ├── e2e/                      # Legacy E2E tests
│   ├── automation/               # Provider automation tests
│   └── load/                     # Load tests
│
├── benchmarks/                   # Performance benchmarks
├── security/                     # Security tests
├── standalone_tests/             # Standalone CLI tests
├── docker/                       # Docker assets and extended
compose
├── scripts/                      # Build and deployment scripts
└── assets/                       # Logo and image assets
```

Verified Real Implementations

AUTH-001: Authentication System (VERIFIED REAL)

File: `internal/auth/auth.go` (~470 lines) **Assessment**: Production-ready

- User registration with validation
- Password hashing with bcrypt + argon2 fallback
- JWT token generation and verification (JWT v4)
- Session management with crypto-random tokens

- Constant-time comparison for timing attack prevention
- Full test coverage in `internal/auth/auth_test.go` (~777 lines)

DB-001: Database Layer (VERIFIED REAL)

File: `internal/database/database.go` **Assessment**: Production-ready

- PostgreSQL connection pool via pgx/v5
- Full schema initialization (users, workers, tasks, projects, sessions, LLM providers, MCP servers, notifications, audit logs)
- `DatabaseInterface` for testability
- Graceful degradation when host is empty

SRV-001: HTTP Server (VERIFIED REAL)

File: `internal/server/server.go` **Assessment**: Production-ready

- Gin-based server with 50+ routes across `/api/v1/`
- JWT auth middleware, CORS, security headers
- WebSocket endpoint for MCP
- Health check with DB + Redis validation
- Graceful shutdown (30s timeout)

LLM-001: LLM Providers (VERIFIED REAL)

File: `internal/llm/` (~5000+ lines across providers) **Assessment**: Real HTTP clients

- `AnthropicProvider` (~752 lines): Full SSE streaming, prompt caching, extended thinking, tool calls
- `OpenAIProvider` (~431+ lines): Full HTTP API client
- `ModelManager`: Multi-provider orchestration, selection strategy, fallback chain
- 16 provider subdirectories with real HTTP implementations
- **Note**: The `internal/llm/` package is genuine. Bluff areas are at `cmd/cli/main.go` only.

WRK-001: Worker Pool (VERIFIED REAL)

File: `internal/worker/` (~800+ lines) **Assessment**: Real distributed worker management

- `WorkerManager`: Register, heartbeat, assign tasks, complete tasks
- SSH config parsing, capability matching, resource tracking
- Health checks with TTL

TSK-001: Task Management (VERIFIED REAL)

File: `internal/task/` (~1000+ lines) **Assessment**: Real task life-cycle

- Priority queues, dependency validation, checkpointing
- Redis caching with graceful degradation
- Retry logic and cleanup

WFL-001: Workflow Engine (VERIFIED REAL)

File: `internal/workflow/` (~1100+ lines) **Assessment**: Real shell execution

- `Executor` dispatches to real `exec.CommandContext()` calls
- Security filtering via `isDangerousCommand()` (rm, dd, mkfs, fork bombs, etc.)
- LLM integration with real `LLMRequest`
- Supports Go, Node, Python, Rust project types

TOO-001: Tools Ecosystem (VERIFIED REAL)

File: `internal/tools/` (~2000+ lines) **Assessment**: Real tool registry

- 8 categories: filesystem, shell, web, browser, mapping, multiedit, confirmation, notebook
- Real chromedp browser automation
- Transactional multi-file editing

EDT-001: Code Editor (VERIFIED REAL)

File: `internal/editor/` (~600+ lines) **Assessment**: Real file I/O

- Diff, whole-file, search/replace, line-based editors
- Automatic file backup with `io.Copy`
- `EditApplier` / `EditValidator` interfaces

NOT-001: Notification Engine (VERIFIED REAL)

File: `internal/notification/` (~800+ lines) **Assessment**: Real HTTP/SMTP calls

- Slack (webhook HTTP POST), Email (SMTP via `net/smtp`), Telegram (Bot API), Discord (webhook)
- Yandex Messenger (OAuth API), Max (enterprise API)
- Rate limiting, retry, queue, metrics

MCP-001: MCP Protocol Server (VERIFIED REAL)

File: `internal/mcp/` (~400+ lines) **Assessment**: Real WebSocket server

- gorilla/websocket concurrent session handling
- JSON-RPC-like message format
- Tool execution dispatch

CFG-001: Configuration Management (VERIFIED REAL)

File: `internal/config/` (~1700+ lines) **Assessment**: Full Viper integration

- Environment variable binding (`HELIX_*`)
- Config file search (`.`, `$HOME/.helixcode`, `/etc/helixcode`)
- Validation rules, default config creation
- `ConfigManager` for load/save/merge

QA-001: HelixQA Integration (VERIFIED REAL)

Files: `internal/helixqa/`, `internal/server/qa_handlers.go`, `applications/terminal-ui/main.go` **Assessment**: Full embedded QA engine with real session lifecycle

- `Engine` struct manages QA sessions with map + sync.RWMutex
- `StartSession()`, `CancelSession()`, `GetSession()`, `ListSessions()` with real state tracking
- REST API: `POST /api/v1/qa/session`, `GET /api/v1/qa/session/:id/status`, `GET /api/v1/qa/session/:id/report`, `GET /api/v1/qa/session/:id/screenshot/:name`, `DELETE /api/v1/qa/session/:id`
- CLI flags: `--qa-run`, `--qa-list`, `--qa-report`, `--qa-screenshot`, `--qa-cancel`
- TUI dashboard with session table, stats panel, refresh/cancel actions
- Screenshot pipeline: 8 platform engines (Linux, Web, iOS, Android, CLI, TUI, macOS, Windows)
- Tests: `internal/helixqa/wrapper_test.go`, `internal/server/qa_handlers_test.go`, `pkg/screenshot/*_test.go`

Verified Bluff & Stub Areas (MUST FIX)

BLUFF-001: LLM Generation is Simulated in Legacy CLI (CRITICAL) — FIXED

File: `cmd/cli/main.go` lines ~236-284 **Evidence**: Previously returned `fmt.Sprintf("Generated response for: %s...", prompt)` without calling any provider. **Fix**: `handleGenerate()` now constructs a real `llm.LLMRequest` with user messages and calls `provider.Generate()` / `provider.GenerateStream()`. Errors are propagated to the user if the provider is unavailable. **Verification**: `go build -tags nogui ./cmd/cli/` compiles; provider call is real (returns error if Ollama/etc. is not running). **Fix Priority**: P0 — RESOLVED

BLUFF-002: Model Listing is Hardcoded in Legacy CLI (CRITICAL) — FIXED

File: `cmd/cli/main.go` lines ~101-128 **Evidence**: Previously only 3 hardcoded models. No dynamic discovery. **Fix**: Replaced with verifier-aware `handleListModels()` that queries LLMsVerifier adapter first, falls back to provider discovery, then to constitutional `FallbackModels` (7 models with scores and verification status). **Verification**: `go test -v ./internal/verifier/...` passes; `go build ./cmd/cli/...` compiles. **Fix Priority**: P0 — RESOLVED

BLUFF-003: Command Execution is Simulated in Legacy CLI (HIGH) — FIXED

File: `cmd/cli/main.go` lines ~310-324 **Evidence**: Previously printed the command and slept for 1 second without executing anything. **Fix**: `handleCommand()` uses `exec.CommandContext(ctx, "sh", "-c", command)` with real `os.Stdout`/`os.Stderr` redirection. Exit codes are reported. **Verification**: `go build -tags nogui ./cmd/cli/` compiles. **Fix Priority**: P0 — RESOLVED

STUB-001: Security Scanning is Simulated

File: `internal/security/security.go` (~132 lines) **Evidence**: `ScanFeature()` contains explicit "Simulate security scanning logic" comment. Always returns `Success=true, Score=95` with empty issues. **Fix Priority**: P1

STUB-002: Memory Redis/Memcached Providers Store Locally

File: `internal/memory/` (~1800+ lines) **Evidence**: `RedisMemoryProvider` and `MemcachedMemoryProvider` store data in local maps with comments like "Redis client would be used in production." Connection config is parsed but not used. **Fix Priority**: P2

STUB-003: Security-Test Entry Point is Entirely Simulated

File: `cmd/security-test/main.go` **Evidence**: Hardcoded list of 12 simulated security tests. `simulateSecurityScan()` returns pre-canned issue lists per category. **Fix Priority**: P2

STUB-004: Several `helix` Subcommands are Print-Only

File: `cmd/other_commands.go` **Evidence**: `server`, `generate`, `test`, `worker`, `notify` commands are stubbed (print placeholder messages). **Fix Priority**: P2

STUB-005: Several `helix-config` Subcommands are Placeholders

File: `cmd/helix-config/main.go` **Evidence**: Many template/history/schema subcommands print placeholder messages. **Fix Priority**: P3

BLUFF-004: LLMsVerifier Integration is Stubbed or Bypassed (CRITICAL)

File Pattern: `internal/verifier/*.go` containing empty structs, `// TODO`, or methods that return hardcoded data instead of calling the verifier. **Evidence**:

- `VerificationService` methods return hardcoded `VerificationResult{OverallScore: 8.5}` instead of querying the verifier database
- `ModelDiscoveryService` returns an empty slice instead of calling provider APIs
- The verifier client returns fallback models without attempting a real HTTP call **Fix Priority**: P0 - Immediate **Verification Command**:

```
make test-verifier-integration
# This MUST pass with real verifier data, not mocked scores
```

BLUFF-005: Provider Discovery Uses Hardcoded Env Var Names (HIGH)

File Pattern: `internal/verifier/startup.go` or provider adapter files containing hardcoded strings like `"OPENAI_API_KEY"` without checking `SupportedProviders[provider].EnvVars`. **Fix Priority**: P1 - High

BLUFF-006: Model Capabilities Are Hardcoded (HIGH)

File Pattern: `internal/llm/*.go` containing `SupportsToolUse: true` as a struct literal for specific models, or `Provider.GetCapabilities()` returning a static slice. **Fix Priority**: P1 - High **Constitutional Impact**: Violates CONST-041 (MCP/LSP/ACP/Embedding/RAG/ Skills/Plugins Integration Mandate).

BLUFF-007: Test Claims Integration But Uses Mocked Verifier (CRITICAL)

File Pattern: `*_test.go` files with `testify/mock` or `testMode: true` in non-unit test files. **Fix Priority**: P0 - Immediate **Constitutional Impact**: Violates CONST-038 (Model Provider Anti-Bluff Guarantee) and CONST-035 (Zero-Bluff Testing).

BLUFF-008: Scoring Weights Do Not Sum to 1.0 (MEDIUM)

File Pattern: `configs/verifier.yaml` or `internal/verifier/config.go` where scoring weights are misconfigured. **Fix Priority**: P2 - Medium

BLUFF-009: `/metrics` Endpoint Returns Hardcoded Zeros (CRITICAL) — FIXED

File: `internal/server/handlers.go` lines ~834-855 **Evidence**: All dynamic metrics (goroutines, memory, database connections) were hardcoded to `0`. **Fix**: `getMetrics()` now calls `runtime.ReadMemStats()`, `runtime.NumGoroutine()`, and `s.db.Pool.Stat()` to return real values. **Fix Priority**: P0 — RESOLVED

BLUFF-010: Multi-Edit Conflict Detection is a No-Op (HIGH) — FIXED

File: `internal/tools/multiedit/transaction.go` lines ~352-369 **Evidence**: `detectFileConflict()` always returned `nil, nil` with comment "For now, we'll assume no conflicts." **Fix**: Implemented real conflict detection — reads the file from disk, computes SHA-256, and compares against the `Checksum` field. Returns `ConflictModified` or `ConflictDeleted` when appropriate. **Fix Priority**: P1 — RESOLVED

Configuration Management

Primary Configuration

Main config at `config/config.yaml`:

```yaml
server:
  address: "0.0.0.0"
  port: 8080
  read_timeout: 30
  write_timeout: 30
  idle_timeout: 300
  shutdown_timeout: 30

database:
  host: ""          # Empty string disables PostgreSQL
  port: 5432
  user: "helix"
  password: "${HELIX_DATABASE_PASSWORD}"
  dbname: "helixcode_prod"
  sslmode: "disable"

redis:
  host: "redis"
  port: 6379
  password: "${HELIX_REDIS_PASSWORD}"
  db: 0
  enabled: true

auth:
  jwt_secret: "${HELIX_AUTH_JWT_SECRET}"
  token_expiry: 86400
  session_expiry: 604800
  bcrypt_cost: 12

workers:
  health_check_interval: 30
  health_ttl: 120
```

```yaml
  max_concurrent_tasks: 10

tasks:
  max_retries: 3
  checkpoint_interval: 300
  cleanup_interval: 3600

llm:
  default_provider: "local"
  max_tokens: 4096
  temperature: 0.7
  timeout: 30
  max_retries: 3
  providers:
    <name>:
      type: <provider-type>
      endpoint: <url>
      enabled: true
      parameters:
        timeout: 30.0
        max_retries: 3
        streaming_support: true
        api_key: ""
  selection:
    strategy: "performance"
    fallback_enabled: true
    health_check_interval: 30

logging:
  level: "info"
  format: "text"
  output: "stdout"

notifications:
  enabled: true
  rules:
    - name: "..."
      condition: "type==error"
      channels: ["slack", "email"]
      priority: urgent
      enabled: true
  channels:
    slack: { enabled, webhook_url, channel, username, timeout }
    telegram: { enabled, bot_token, chat_id, timeout }
    email: { enabled, smtp: { server, port, username, password,
        tls }, recipients, timeout }
    discord: { enabled, webhook_url, timeout }
```

Environment Variables

Required for Production:

- `HELIX_DATABASE_PASSWORD`
- `HELIX_AUTH_JWT_SECRET`
- `HELIX_REDIS_PASSWORD`

LLM Provider Keys (as needed):

- `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, `XAI_API_KEY`, `DEEPSEEK_API_KEY`, `GROQ_API_KEY`, `MISTRAL_API_KEY`, `COHERE_API_KEY`, `AZURE_OPENAI_API_KEY`, `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`

Notification Integrations:

- `HELIX_SLACK_WEBHOOK_URL`
- `HELIX_TELEGRAM_BOT_TOKEN`, `HELIX_TELEGRAM_CHAT_ID`
- `HELIX_EMAIL_SMTP_SERVER`, `HELIX_EMAIL_USERNAME`, `HELIX_EMAIL_PASSWORD`
- `HELIX_DISCORD_WEBHOOK_URL`

Testing Strategy

Test Categories

1. **Unit tests**: Mocks allowed, `*_test.go`, `-short` flag
2. **Contract tests**: Real API schemas, no mocks
3. **Component tests**: Real subsystems wired together
4. **Integration tests**: Full app with real dependencies (`-tags=integration`)
5. **E2E challenges**: Complete user workflows against real LLM APIs
6. **Security tests**: OWASP compliance
7. **Performance tests**: Benchmarks
8. **Automation tests**: Provider/hardware automation (`-tags=automation`)
9. **Load tests**: Stress testing

Anti-Bluff Testing Rules

- Unit tests: Mocks OK
- **ALL other tests: Real infrastructure ONLY**
- Every PASS guarantees **Quality + Completion + Usability**
- Challenges fail on simulated/stubbed behavior
- No bare `t.Skip()` without `SKIP-OK: #<ticket>` marker

Docker Test Infrastructure

- `docker-compose.test.yml`: PostgreSQL 16, Redis 7, Memcached, Cognee, ChromaDB, Qdrant, Ollama, Prometheus, Grafana
- `docker-compose.full-test.yml`: Complete stack with mock-LLM server, Selenium, ChromeDP, SSH server + 3 workers, Cognee, Weaviate, mock-Slack, multicast router

Challenge Framework (`tests/e2e/challenges/`)

The most rigorous test system validates HelixCode by having it **generate real projects** and testing them:

- **Challenge Definitions**: JSON specs (ASCII art generator, CLI task manager, JSON validator, notes API, tic-tac-toe TUI, URL shortener)
- **Execution Flow**: Load spec → Call real LLM API → Parse generated code → Compile → Test → Runtime validation
- **Validation Layers**: Directory structure, code quality, compilation, testing, functionality, runtime validation with diverse data
- **Test Matrix**: Supports CLI, TUI, REST, WebSocket interfaces across 15+ providers and worker pool distributions

Test Scripts Summary

```
# Basic
cd HelixCode && make test

# Full infrastructure (recommended for validation)
make test-infra-up
make test-complete
make test-infra-down

# Individual categories
make test-unit-full
make test-integration-full
make test-e2e-full
make test-security-full
make test-load-full

# Legacy scripts
./run_tests.sh --all
./run_all_tests.sh
./run_integration_tests.sh
```

Docker Deployment

Production (`docker-compose.yml`)

Services: helixcode-server (8080, 2222), postgres:15, redis:7, nginx (80, 443), prometheus (9090), grafana (3000)

Quick Start

```
cd HelixCode
cp .env.example .env
# Edit .env with secure passwords
docker compose up -d
docker compose ps
curl http://localhost/health
```

Other Compose Files

File	Purpose
`docker-compose-simple.yml`	Minimal dev (postgres + redis only)
`docker-compose.test.yml`	Integration/E2E testing stack
`docker-compose.full-test.yml`	Zero-skip full test infrastructure
`docker-compose.aurora-os.yml`	Security-focused Aurora OS platform
`docker-compose.harmony-os.yml`	Distributed Harmony OS platform
`docker-compose.specialized-platforms.yml`	Combined Aurora + Harmony
`docker/docker-compose.yml`	Extended full-stack with Milvus, Elasticsearch, MLflow, Jaeger, Jupyter, Portainer

Deployment Patterns

- Healthchecks on every service
- Docker profiles: `monitoring`, `distributed`, `with-redis`, `production`, `dev`, `server`
- Isolated bridge networks per deployment
- Named persistent volumes for all stateful services
- `.env` file for secrets

Code Style & Development Conventions

Go Conventions

- Standard Go formatting: `go fmt ./...`
- Linting: `golangci-lint run ./...` (timeout 10m in CI)
- Vet: `go vet ./...`
- Table-driven tests with `t.Run()` subtests
- Build tags for integration/automation tests: `//go:build integration`

Project Conventions

- **Always work from `helix_code/` subdirectory**
- **Generate logo assets before first build**: `make logo-assets`
- **Database/Redis optional**: Disable by setting `database.host: ""`
- **Environment variables override config file**
- Use `internal/` for all core packages; no `pkg/` directory in active use
- Error handling: explicit, no silent failures
- Concurrent access: use `sync.RWMutex` or channel patterns

API Conventions

- REST API documented in `api/openapi.yaml` (OpenAPI 3.0.3)
- Base path: `/api/v1`
- Authentication: Bearer JWT via `Authorization` header
- Health endpoint: `GET /health` (no auth required)

Security Considerations

Verified Security Features

- Password hashing: bcrypt (cost 12) with argon2 fallback
- JWT with constant-time comparison
- CORS middleware, security headers (X-Frame-Options, CSP, HSTS)
- Rate limiting support in production config
- Session timeout, concurrent session limits, IP binding options
- Workflow `isDangerousCommand()` filter blocks rm, dd, mkfs, fork bombs, etc.
- Input validation in auth and server packages

Security Testing

- `security/security_test.go`: OWASP Top 10, SAST, DAST, credential scanning, TLS enforcement, input validation (path traversal, XSS, SQL injection, command injection, SSRF)
- File permission checks (0600 for configs)

Known Security Stubs

- `internal/security/security.go`: Simulated scanning (always returns clean)
- `cmd/security-test/main.go`: Entirely simulated security tests

Production Hardening

- Use `HELIX_AUTH_JWT_SECRET` with high entropy
- Enable PostgreSQL SSL in production
- Enable Redis authentication
- Configure CORS `allowed_origins` explicitly
- Enable audit logging
- Set `bcrypt_cost: 14` in production

Universal Mandatory Constraints

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines** (Note: existing workflow files in `.github/workflows/` are legacy and must not be expanded)
2. **NO HTTPS for Git** (SSH only)
3. **NO manual container commands** (orchestrator-owned)

Mandatory Development Standards

1. **100% Test Coverage** (unit, integration, E2E, automation, security, benchmark)
2. **Challenge Coverage** (every component)
3. **Real Data** (actual API calls, real DB, live services)
4. **Health & Observability** (health endpoints, circuit breakers)
5. **Documentation & Quality** (update docs with code changes)
6. **Validation Before Release** (full suite + all challenges)
7. **No Mocks in Production**
8. **Comprehensive Verification** (runtime, compile, structure, dependencies, compatibility)
9. **Resource Limits** (30-40% of host resources max)
10. **Bugfix Documentation** (root cause, affected files, fix, verification link)
11. **Real Infrastructure for All Non-Unit Tests**

12. **Reproduction-Before-Fix** (Challenge first, then fix)
13. **Concurrent-Safe Collections**

Definition of Done

A change is NOT done because code compiles. "Done" requires:

- Pasted terminal output from a real run
- No self-certification words without evidence
- Demo commands that run against real artifacts
- Loud skips with `SKIP-OK: #<ticket>` markers

CONST-035 — End-User Usability Mandate

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work — buggy challenge wrappers masked failed assertions, scripts checked file existence without executing the file, "reachability" tests tolerated timeouts, contracts were honest in advertising but broken in dispatch. **This MUST NOT recur in HelixCode.**

Every PASS result MUST guarantee: a. **Quality** — correct behavior under real inputs, edge cases, concurrency b. **Completion** — wired end-to-end with no stub/placeholder gaps c. **Full usability** — a user following documentation succeeds

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

Bluff Taxonomy (each pattern observed and now forbidden)

- **Wrapper bluff** — assertions PASS but wrapper's exit-code logic is buggy
- **Contract bluff** — system advertises capability but rejects it in dispatch
- **Structural bluff** — file exists but doesn't contain working code
- **Comment bluff** — comment promises behavior code doesn't have
- **Skip bluff** — `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker

The taxonomy is illustrative, not exhaustive. Every Challenge or test added going forward MUST pass an honest self-review against this taxonomy before being committed.

Constitutional anchors (cascaded from `CONSTITUTION.md`)

Article XI §11.9 — Anti-Bluff Forensic Anchor

> Verbatim user mandate: "*We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!*"
>
> Operative rule: **The bar for shipping is not "tests pass" but "users can use the feature."** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

Article XII §12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak is a release blocker until rotated and post-mortemed.

Article XII §12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main`/`master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval per operation. Authorization for one push does not extend further. Bypassing hooks / signing / protected-branch rules also requires explicit approval.

CONST-036: LLMsVerifier Single Source of Truth Mandate

Rule: LLMsVerifier SHALL BE the sole authoritative source for:

1. All model metadata (names, IDs, context windows, capabilities)
2. All provider metadata (endpoints, auth types, supported models)
3. All verification status (verified, partial, failed, pending)
4. All scoring data (overall scores, capability scores, tier rankings)

Prohibition: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

Anti-Bluff Verification:

- Challenge script `challenges/scripts/verifier_hardcode_check.sh` scans all Go source files for hardcoded model arrays.
- The only permitted hardcoded data is the 7-entry fallback list in `internal/verifier/fallback_models.go`.

CONST-037: Model Provider Anti-Bluff Guarantee

Rule: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last 24h. Models older than this MUST display a "stale" indicator and be deprioritized.

Anti-Bluff Testing:

- Unit tests MAY mock the verifier client.
- Integration tests MUST start the verifier server and perform real provider discovery.
- The Makefile target `make test-verifier-integration` MUST exist and run without mocks.

CONST-038: Real-Time Model Status Accuracy

Rule: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by LLMsVerifier within 60 seconds.

Polling vs. Push:

- If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier at most every 60s.
- The TUI MUST display a "last updated" timestamp with every model listing.
- Models in "cooldown" or "rate-limited" state MUST show the estimated recovery time if known.

CONST-039: All Providers and Models Integration Mandate

Rule: HelixCode MUST integrate with ALL providers that LLMsVerifier supports, subject only to:

1. The provider being explicitly disabled in configuration (`enabled: false`)
2. The API key being absent and the provider requiring one
3. The provider being marked `deprecated` in the verifier database

Minimum Provider Set (SHALL NOT be reduced without constitutional amendment): OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp.

CONST-040: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

Rule: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

1. **MCP**: The verifier MUST report which models support MCP tool calling.
2. **LSP**: The verifier MUST report code-analysis capabilities.
3. **ACP**: The verifier MUST report multi-agent coordination support.
4. **Embedding**: The verifier MUST report `supports_embeddings` for each model.
5. **RAG**: The verifier MUST report context-window sizes for chunking strategies.
6. **Skills / Plugins**: The verifier MUST track plugin compatibility.

Prohibition: Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities()` method MUST return data sourced from the verifier's `VerificationResult` fields.

Free AI Providers

- **XAI (Grok)**: `grok-3-fast-beta`, `grok-3-mini-fast-beta`
- **OpenRouter**: Free models from various providers
- **GitHub Copilot**: `gpt-4o`, `claude-3.5-sonnet` (with subscription)
- **Qwen**: 2,000 requests/day free tier

Host Power Management — Hard Ban (CONST-033)

Host Power Management is Forbidden.

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to every shell command, script, container entry point, systemd unit, test, CLI suggestion, snippet, or example you emit.

Common Issues

1. **Build fails**: Run `make logo-assets` then `make build`
2. **Database errors**: Check `HELIX_DATABASE_PASSWORD`
3. **Worker SSH failures**: Verify SSH key authentication
4. **LLM timeouts**: Check provider status and config
5. **Redis connection failures**: Check `HELIX_REDIS_PASSWORD` and `redis.enabled`
6. **Test skips**: Ensure `SKIP-OK: #<ticket>` marker is present for any intentional skips

---

Resources & References

- **Constitution**: `CONSTITUTION.md`
- **CLAUDE.md**: `CLAUDE.md`
- **Gap Analysis**: `HELIXCODE_GAP_ANALYSIS.md`
- **Zero-Bluff Plan**: `HELIXCODE_ZERO_BLUFF_PLAN.md`
- **Testing Strategy**: `ANTI_BLUFF_TESTING_STRATEGY.md`
- **OpenAPI Spec**: `helix_code/api/openapi.yaml`
- **Docker Guide**: `helix_code/DOCKER_DEPLOYMENT.md`

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end- user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 (positive-evidence-only validation) + §11 (bleeding-edge ultra-perfection quality bar) + §11.3 (the "no bluff" CLAUDE.md / AGENTS.md mandate) + **§11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-PROPAGATION`)**.

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Per parent Constitution §11.4.1, every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh` / `action_timeline.sh` / Go `recording-analyzer` / `helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display`-based dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re- discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then**: (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff

gate. Tests AND HelixQA Challenges are bound equally — Challenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available `superpowers:*` skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator- wired in `test_all_fixes.sh`), HelixQA test bank entry (`banks/atmosphere.yaml` + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: `docs/Issues.md` → `docs/Fixed.md` migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (`docs/guides/*.md`), affected diagrams/flowcharts/architecture docs, per-version `docs/changelogs/<tag>.md` entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: `meta_test_false_positive _proof.sh` returns all gates green AND every gate's paired mutation FAILs (no bluff gates); `docs/Issues.md` open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

> "We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSes because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSes MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer underrun-overrun-partial / AudioFlinger writeError counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSes MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (`Application not responding`, `Force close`, sign-in dialog, geo-restriction overlay, ad break, paywall, `App is not certified`). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = `tinycap` + `aplay --dump-hw-params` + `ffprobe` + `/proc/asound` parsers (`lib/audio_validation.sh` per §11.2.5). Video = `screenrecord` + `ffprobe -count_frames` + `recording-analyzer` + Tesseract OCR (`scripts/dual_display_record.sh`

- `cmd/recording-analyzer/` per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

MANDATORY §12 HOST-SESSION SAFETY — INCIDENT #2 ANCHOR (2026-04-28)

Second forensic incident: on 2026-04-28 18:36:35 MSK the user's `user@1000.service` was again SIGKILLed (`status=9/KILL`), this time WITHOUT a kernel OOM kill (systemd-oomd inactive, `MemoryMax=infinity`) — a different vector than Incident #1. Cascade killed `claude`, `tmux`, the in-flight ATMOSphere build, and 20+ npm MCP server processes. Likely cumulative cgroup pressure + external watchdog.

Mandatory safeguards effective 2026-04-28 (full text in parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12 Incident #2):

1. `scripts/build.sh` MUST source `lib/host_session_safety.sh` and call `host_check_safety` BEFORE any heavy step.
2. `host_check_safety` has 7 distress detectors including conmon cgroup-events warnings (#6) and current-boot session-kill events (#7).
3. Containers MUST be clean-slate destroyed + rebuilt after any suspected §12 incident. `mem_limit` is per-container, not per-user-slice — operator MUST cap Σ `mem_limit` ≤ physical RAM − user-session overhead.
4. 20+ npm-spawned MCP server processes are a known memory multiplier; stop non-essential MCPs before heavy ATMOSphere work.
5. **Investigation: Docker/Podman as session-loss vector.** Per-container cgroups don't prevent cumulative user-slice pressure; conmon `Failed to open cgroups file: /sys/fs/cgroup/memory.events` warnings preceded the 18:36:35 SIGKILL by 6 min — likely correlated.

This directive applies to every owned ATMOSphere repo and every HelixQA dependency. Non-compliance is a Constitution §12 violation.

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

> "We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLs on 2026-04-30 during 1.1.5-dev — every one happened while `scripts/build.sh` was running `m -j5` AOSP. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down `user@1000.service`. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active CONSTRAINT on heavy work itself.

Mandatory protections (rock-solid):

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60 in `scripts/lib/host_session_safety.sh`.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`.
3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=…`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the safe `-j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate `CM-MEMBUDGET-METATEST` locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §12.6.

Non-compliance is a release blocker regardless of context. *Built with zero-bluff commitment. Every feature actually works.*

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, or `appears to` when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (logcat, dmesg, /sys readings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

Pre-build gate `CM-NO-GUESSING-MANDATE` greps recently-modified docs

- test scripts for the forbidden vocabulary outside explicit `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` blocks. Paired mutation introduces a `likely` token into a fresh status block → gate FAILs. Propagation gate `CM-COVENANT-114-6-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.
"runtime drift"	Label for "we don't know what changed".
"intermittent" / "transient"	Label for "we don't know how to reproduce".
"pending stress retest"	Defers the actual investigation indefinitely.
"correlates with X"	Hypothesis presented as causation.

Pre-build gate `CM-DEMOTION-EVIDENCE-RULE` scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit `UNCONFIRMED:` / `UNATTRIBUTED:` / `PENDING_CYCLE_RETEST:` blocks. Propagation gate `CM-COVENANT-114-7-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix's commit / Issues.md / Fixed.md entry MUST cite at least one external source URL OR the literal "NO external solution found — original work".

Pre-build gate `CM-RESEARCH-CITATION-PRESENT` scans new fix-direction blocks for the pattern. Propagation gate `CM-COVENANT-114-8-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to `docs/guides/` a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that DO NOT require runtime on-device validation to design MUST be landed BEFORE the next firmware rebuild. Anti-pattern eliminated: `Fix A → rebuild → flash → cycle → fix B → rebuild → ...` serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as `REQUIRES_REBUILD: <reason>`: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate `CM-COVENANT-114-9-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (`.example` suffix). Tests load credentials at runtime from `scripts/testing/secrets/` (or per-submodule equivalent); operator-populated files are `chmod 600`,

directory is `chmod 700`. `.env`, `.env.*`, `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen- recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (`.netflix.env`, `.disney.env`, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local `.env`, audit captured artifacts. Pre-build gate `CM-CREDENTIAL-LEAK-SCAN` greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate `CM-COVENANT-114-10-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start` / `cmd media_session play` / `MediaController.play` MUST issue matching `am force-stop` / `input keyevent KEYCODE_MEDIA_STOP` + register cleanup in `EXIT` trap. Verified via positive evidence (Arvus codec-state → `N.E.`, `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.14. Pre-build gates `CM-TEST-PLAYBACK-CLEANUP` + `CM-COVENANT-114-14-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Fixed (→ Fixed.md)`. Status MUST be updated as the item progresses through its lifecycle. `Fixed` requires captured-evidence per §11.4.5 + migration to Fixed.md.

The auto-generated docs/Issues_Summary.md includes the Status column. All three file types (.md, .html, .pdf) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.15. Pre-build gates CM-ITEM-STATUS-TRACKING + CM-COVENANT-114-15-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in docs/Issues.md carries a **Type:** line with one of three values: Bug (product defect / regression / user-visible broken behaviour), Feature (new capability not previously offered to end users), Task (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated docs/Issues_Summary.md includes the Type column. All three file types (.md, .html, .pdf) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.16. Pre-build gates CM-ITEM-TYPE-TRACKING + CM-COVENANT-114-16-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at http://192.168.4.185/), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: scripts/testing/lib/arvus_probe.sh, scripts/testing/arvus_probe.sh, docs/guides/ARVUS_HDMI_INTEGRATION.md. Pre-build gate CM-ARVUS-EVIDENCE-INTEGRATED (7 invariants) + paired mutation. No hardcoding (env: ARVUS_HOST etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.13. Integration reference: docs/guides/ARVUS_HDMI_INTEGRATION.md.

Non-compliance is a release blocker regardless of context.

§11.4.11 — File-layout discipline (User mandate, 2026-05-12)

Files live in canonical directories per type:

- Shell scripts → scripts/ (legacy: scripts/legacy/)
- Log files → logs/ (legacy: logs/legacy/)
- Release artifacts → releases/<app>/<version>/
- Operator credentials → scripts/testing/secrets/ (per §11.4.10, git-ignored)
- Markdown docs → docs/ + docs/guides/ + docs/research/ + docs/superpowers/plans/
- Per-version changelogs → docs/changelogs/
- Hardware ID photos → docs/hardware/<device-slug>/

Repo root contains ONLY: AOSP-mandated top-level files (Android.bp, Makefile, bootstrap.bash, BUILD, kokoro, lk_inc.mk, OWNERS, version_defaults.mk), project metadata (README/CLAUDE/AGENTS/CONTRIBUTING/LICENSE/NOTICE/VERSION), dot-files (.gitignore/.gitmodules), and standard top-level dirs (build/, device/, external/, frameworks/, hardware/, kernel-5.10/, packages/, prebuilts/, scripts/, system/, tools/, vendor/, docs/, releases/, logs/).

NO bash scripts in repo root except AOSP-mandated bootstrap.bash. NO log files in repo root. NO duplicate filenames between root and scripts/. NO release artifacts in root. Moves require triple-verification (audit all references + distinguish absolute vs subdir-local + confirm no AOSP build- system requirement). Pre-build gate CM-FILE-LAYOUT-DISCIPLINE enforces. Propagation gate CM-COVENANT-114-11-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.11.

Non-compliance is a release blocker regardless of context.

§11.4.12 — Issues_Summary.md sync mandate (User mandate, 2026-05-12)

docs/Issues_Summary.md is the canonical short-form summary of all open items. MUST be regenerated + re-exported (HTML + PDF) whenever Issues.md changes. Generator: scripts/testing/generate_issues_summary.sh. Pre-build gates `CM-ISSUES-SUMMARY-SYNC` + `CM-COVENANT-114-12-PROPAGATION` enforce mechanically.

Sort order (User mandate refinement 2026-05-12): severity DESC (C → M → L), then intra-group criticality DESC inside each group. Most critical row = #1, least critical = #N. Documented at the top of the generated file.

Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` — runs generator + `export_progress_docs.sh` in one shot. MUST be invoked after any edit to Issues.md or Issues_Summary.md. HTML+PDF exports are NEVER manually invoked; they ALWAYS travel with the markdown.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.12.

Non-compliance is a release blocker regardless of context.

Submodule Decoupling & Reusability — MANDATORY for ALL AI Agents

Applies to ALL CLI agents (Codex, Cursor, Gemini CLI, Copilot CLI, Claude Code, etc.) working in this repository.

This repository is **shared infrastructure** consumed by multiple independent consumer projects. The value of this repository depends on staying fully decoupled and reusable.

Hard rules:

- DO NOT hardcode any specific consumer project's name, platform list, paths, version strings, or release-naming conventions.
- DO NOT import / reference any consumer-project namespace.
- DO NOT embed consumer-project-specific governance, branding, or rule numbering in `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
- DO assume N ≥ 2 unrelated consumer projects exist.

Cross-project rules MUST be phrased generically ("every consuming project's full platform matrix"), never with a specific consumer's matrix hardcoded.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): "*Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!*"

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12–48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.37 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no `--skip-full-retest` or `--quick-release` flag exists.

Pre-build gate `CM-FULL-SUITE-RETEST-MANDATE` + paired mutation. Propagation gate `CM-COVENANT-114-40-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule `Constitution.md` §11.4.37.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote

`HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate `CM-COVENANT-114-41-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.41.

Non-compliance is a release blocker regardless of context.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1)

anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset

names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): "*naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!*"

Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference

(configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition: the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): "*every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!*"

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`,

`node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): "*We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone.*"

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): "*Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!*"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-

pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): "*Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!*"

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already supports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): "*Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.*"

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate.

Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

Canonical authority: `HelixConstitution/Constitution.md` §7.1 + §11.4.

Forensic anchor — verbatim operator mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

Every PASS MUST carry positive runtime evidence. Consuming-project-specific requirements are defined by each consuming project's Constitution. This submodule's rules remain project-agnostic.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18): "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: (a) programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + structured JSON result file); (b) headless intent dispatch + state poll (`am start --es` / `am broadcast` + `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27.

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite tracked work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence + paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation coverage), §11.4.27 (no-fakes + 100% type coverage), §11.4.39 (per-feature on-device end-user validation), §11.4.43 (TDD RED-first), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach), §11.4.50 (deterministic consistency), §11.4.51 (live-ADB-first).

Pre-build gates: `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule Constitution.md §11.4.52.

Non-compliance is a release blocker regardless of context.

CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

> Verbatim user mandate (2026-05-17): "*make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!*"

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

Verification artefact: every governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section capturing fetch output, per-remote divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

Cascade requirement: This anchor (verbatim or by `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule `Constitution.md` §11.4.41 for the full mandate.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z): "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, executable, emits markdown table with `Status` + `Type` columns per §11.4.19 column-alignment). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH summaries in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: `CM-FIXED-SUMMARY-SYNC` (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule Constitution.md §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential Phase-chain. Each PWU has: ATM-NNN identifier (§11.4.54), Issues.md entry (§11.4.15+§11.4.16), file-scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired §1.1 meta-test mutation, HelixQA Challenge bank entry, captured-evidence directory (§11.4.5+§11.4.52).

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in worktrees) → Stage 2 MERGE (serial conductor + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N.

Synchronization: 4-layer lock hierarchy (parent flock / per- submodule git / contention-path advisory locks for 10 forbidden cross-PWU paths / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four): C1 §11.4.43 RED-test captured. C2 §1.1 paired meta-test mutation FAILs the gate. C3 §11.4.50 3-iter (or 10-iter) deterministic-consistency. C4 §11.4.5 captured-evidence per feature type.

Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS REJECTED. HelixQA Challenge bank coverage MANDATORY for every user- visible PWU.

Phase 39.EX infrastructure gates (5 gates land the parallel infrastructure itself): `CM-PWU-PARALLEL-VALIDATION-ORCHESTRATOR`, `CM-PWU-HELIXQA-PER-DOMAIN-RUNNER`, `CM-PWU-WORKER-POOL-LOCKING`, `CM-PWU-FILE-SCOPE-PARTITION`, `CM-PWU-AUTO-MERGE-GATE-6CONDITIONS`. Each ships a paired meta-test mutation per §1.1.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE`

- `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION`. Paired mutations cover each gate. No escape hatch.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference: `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA submodules' equivalents. Excludes: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-5.10/**`, `out/**`, `build/**`, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be ≥ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION`. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!"

- "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "${BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee …) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules). OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3)

shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

> Verbatim user mandate (2026-05-20): "*We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!*"

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never SKIP, never PASS. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

Cascade requirement: This anchor (verbatim or by `§11.4.68` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.68 for the full mandate.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids

`network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

> Verbatim user mandate (2026-05-20): "*Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!*"

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by `§11.4.70` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

> Verbatim user mandate (2026-05-20): "*before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!*"

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`--ours`/`--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer cover-

age + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote` post-push. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.71` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

> Verbatim user mandate (2026-05-20): "*Make sure all fixes for audio are always top priority in main working stream!*"

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by `§11.4.72` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

> Verbatim user mandate (2026-05-20): "*Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!*"

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table `Revision` integer, bumped for every other change); (3) the metadata table MUST stay current (`Revision`, `Last modified`, `Status summary`, `Fixed`); (4) propagated copies of the rule MUST reference the active `specification.V<primary>.md`, not a stale archive; (5) on primary bump the old file moves to `<spec-dir>/archive/` with `Status: superseded`. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by `§11.4.73` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule `Constitution.md` §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

> Verbatim user mandate (2026-05-20): "*We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!*"

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — `vasic-digital` and `HelixDevelopment` on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a `Catalogue-Check:` field (`reuse <org/repo>@<sha>` / `extend <org/repo>@<sha>` / `no-match <date>`). Classification: universal.

Cascade requirement: This anchor (verbatim or by `§11.4.74` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule `Constitution.md` §11.4.74 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Forensic anchor — direct user mandate (verbatim, 2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in this project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock; per-iteration latency p50/p95/p99 recorded) + concurrent contention (N ≥ 10 parallel invocations; no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input; every boundary produces a categorised result, never an uncaught exception).

Chaos (closed-set, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call; categorised recovery) + network-fault injection (drop/delay/reorder; `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test; detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion; refuse cleanly OR degrade gracefully — NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault; recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS cites a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT`; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (stress + chaos test files exist + executable + parseable under sh -n + bash -n per §11.4.67; helper library exists; the fix's pre-build gate cites the stress + chaos test file path) + paired meta-test mutation per §1.1 (stripping chaos-injection or per-iteration evidence capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51, dispatched against real device, evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden), §11.4.5 (captured-evidence quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists.